# AUB Course Planner: An MCP Toolkit

## C5 Agent Tools: Part 1 - MCP Concepts and Server Critique

Nanar Aintablian
nsa81@mail.aub.edu

Fall 2026–2027

## Key MCP Concepts

LLMs are capable but isolated from live data and tools; before MCP, every connection between an AI application and an external service was custom-built, making integrations brittle and hard to scale [1]. Anthropic released the Model Context Protocol in November 2024 as an open, model-agnostic standard for how AI applications discover and use external capabilities. Three roles communicate over JSON-RPC 2.0:

- **Host:** the application where the agent runs and reasons.
- **Client:** a connector inside the host that keeps a one-to-one session with a single server.
- **Server:** runs outside the host and exposes **tools** (functions the model invokes), **resources** (data the application reads) and **prompts** (templates the user selects).

A request flows Agent → Client → Server → external system and back. Because clients discover capabilities at runtime, new servers plug in without changing agent code, reducing an $N \times M$ integration problem to $N + M$. MCP is an integration layer, not an orchestrator: frameworks decide *when* to call a tool, while MCP standardizes *how*. The article also lists limitations: server-management overhead, still-maturing security, and no guarantee that more tools are used well.

## Critique of the Fetch MCP Server

Fetch (v2026.10.24) exposes one tool, `fetch`, which retrieves a URL and returns its content as Markdown, plus a matching `fetch` prompt for user-initiated requests. Measured against the class tool design principles, it does well:

- **Isolated functionality:** it only retrieves; summarizing and reasoning are left to the model.
- **Well-defined API:** one required argument (`url`) and three optional, typed ones with documented defaults (`max_length`, `start_index`, `raw`). Its annotations (read-only, non-destructive, idempotent) document its side effects.
- **Predictable failure:** robots.txt refusals, unreachable pages, HTTP errors and blocked private addresses return a result flagged `isError` with a readable reason, not a silent empty response.

Weaknesses:

- **No output schema (API):** results are plain text, so when the model pages through long content with `start_index`, no structured field tells it how much remains.
- **Hidden behavior (predictability):** robots.txt compliance, the user-agent and private-address access depend on launch flags (`-ignore-robots-txt`, `-user-agent`, `-allow-private-ips`), and robots.txt applies to tool calls but not prompt calls. None of this appears in the schema, so identical calls can behave differently across deployments.

Overall, Fetch is a clean example of a pure *Data* tool: one job, read-only, with explicit failures.

## References

[1] K. Se, "What Is MCP? Model Context Protocol in Agentic AI, Explained," Hugging Face Blog, 2025. https://huggingface.co/blog/Kseniase/mcp